

PERGAMON

Neural Networks 11 (1998) 1551–1569

Neural Networks

1998 Special Issue

Embedded neural networks: exploiting constraints

Christian Scheier[a,*], Rolf Pfeifer[a], Yasuo Kunyioshi[b]

[a]*Artificial Intelligence Laboratory, University of Zurich, Winterthurerstrasse 190, 8057 Zurich, Switzerland*
[b]*Humanoid Interaction Laboratory and Intelligent Systems Division, Electrotechnical Laboratory, Tokyo, Japan*

Received and accepted 14 May 1998

Abstract

Using concepts and tools of embodied cognitive science, we investigate the implications of embedding neural networks in a physical structure, the body of a robot. Embedding a neural network in a body provides constraints that can be exploited for learning. We show that the constraints are given by the environment and object properties, the agent's morphology, the agent's motor system and specific ways of interacting with the objects. We argue that designing embedded neural networks implies (a) understanding these constraints, and (b) exploiting them, i.e., designing neural networks such that they—one way or other—incorporate the constraints. This in turn results in cheap and simple networks that are suited for the task environment, and have real-time responses. Moreover, this constraint-based approach provides new perspectives on two fundamental problems of cognitive science: focus-of-attention and object constancy. The main arguments are illustrated with a series of case studies with simulated and physical mobile robots that are controlled by hand-designed as well as evolved neural networks. © 1998 Elsevier Science Ltd. All rights reserved.

Keywords: Embodiment; Constraints; Learning; Adaptive behavior

1. Introduction

Recently, a new paradigm of cognitive science has emerged. The hallmark of this new approach is the focus on the situated and embodied nature of intelligence. Research in so-called behavior-based artificial intelligence (Arkin, 1998 Brooks, 1991), embodied neurobiology (Chiel and Beer, 1997 Almassy et al., in press), and embodied cognitive science (Ballard et al., 1997 Clark, 1997 Pfeifer and Scheier, in press-a) has challenged the traditional view according to which intelligence is an abstract, symbolic process independent of physical implementation. The embodied approach to intelligence is in accordance with recent empirical findings in a number of disciplines such as developmental psychology (e.g., Thelen and Smith, 1994), cognitive psychology (e.g., Glenberg, 1997), or the neurosciences (e.g., Salinas and Romo, 1998). It has been shown, for example, that people tend to minimize memory storage in natural tasks by exploiting their eye movements (e.g., Bensinger et al., 1995), that individuals are oblivious to significant changes in their visual field and thus are unlikely to store detailed sensory information in a sensory buffer (a phenomenon called 'change blindness', Simons and Levin, 1997) or that blindfolded subjects which are asked to mentally rotate previously seen objects by 90 degrees have a much higher accuracy of pointing in the right direction when they are allowed to actually rotate 90 degrees, compared to pointing accuracy when only mental rotation is involved (Montello and Presson, 1993). These and other results are the beginning of a new line of research that focuses on the coupling between brain, body and environment.

Chiel and Beer (1997) have recently summarized the main implications of this embodied view in the context of biological neural networks (i.e., animal brains):

1. filtering: sensory preprocessing and motor post-processing filter inputs to and outputs from an agent's nervous system;
2. matching: nervous system and periphery both co-evolve and co-develop, resulting in matching and complementarity between them;
3. constraints and opportunities: body structure imposes constraints and offers opportunities for neural control;
4. feedback: continuous feedback between nervous system, body, environment are critical for normal, adaptive behavior.

In this paper we focus, from a synthetic perspective, on the third point in this list. Our goal is to show how embedded neural networks are constrained by and can

* Corresponding author. Tel.: +41-1-236-4343; Fax: +41-1-363-00-25; E-mail: scheier@ifi.unizh.ch

0893–6080/98/$19.00 © 1998 Elsevier Science Ltd. All rights reserved.
PII: S0893-6080(98)00084-7

take advantage of an agent's body structure in a number of learning tasks some of which are not solvable by traditional (non-embodied) networks.

This focus does not deny the importance of the other implications summarized by Chiel and Beer (1997), however. Matching, for instance, between nervous system and body surface is as important for synthetic agents as it is for natural agents. Consider the robot shown in Fig. 1a. It is a simple robot equipped with a number of infrared (IR) sensors. It is controlled by a very simple neural network that implements the following rule: if there is sensory stimulation on the left turn right (and vice versa), a rule intended for obstacle avoidance. If several of these robots are put into an arena with styropor cubes, they move the cubes into clusters, and some cubes end up along the wall (Fig. 1b). The reason is given in Fig. 1c. Normally the robots simply avoid obstacles. If they happen to encounter a cube head on, they push the cube. However, they are not searching for cubes and then pushing them: because of the particular geometry and the arrangement of the sensors, they push the cubes if they happen to encounter them in the appropriate direction. They push the cube until there is another cube on the side that will provide sufficient sensory stimulation to cause the robot to turn away. But now there are already two cubes together and the probability that an additional cube will be deposited near them is increased. It is increased because the environment has changed, not because something has changed inside the robot: These robots are purely reactive.

If now the position of the IR sensors is slightly changed as shown in Fig. 1d, the robots will no longer move the cubes into clusters. The main point to be noted is that the robots are still controlled by the same nervous system but their behavior has drastically changed because of the position change of the sensors. In other words, the same network can result in completely different behaviors if its embedding is changed. In the example, the embedding was changed by moving the two front sensors closer to each other, but leaving everything else identical. The point is, of course, that this network is embedded in and matched to a particular body structure. In other words, if we are to understand the function of the network itself, we have to study not only the neuronal dynamics and structures, but also their relation to the body. These and related aspects of embodiment have not been taken into account in the neural networks literature, but, we argue, need to be investigated if we are to understand learning and adaptive behavior in general. The goal of this paper is to present first steps toward an understanding of embedded networks. We focus on two aspects: constraints and opportunities that can be derived from embodiment, i.e., from embedding a neural network in a body. The two are intimately coupled. On the one hand, constraints can be turned into opportunities for learning because exploiting constraints in the design of a network results in cheaper and smaller networks. In a more general form this point has been made by Ashby more than 40 years ago: ''...when a constraint exists advantage can usually be taken of it.'' (Ashby, 1956, p. 130). On the other hand, opportunities such as the possibility to interact with objects can impose constraints on the data the network has to learn.

The structure of this paper is as follows. We start by conceptualizing the implications of embedding a network in an autonomous agent (Section 2). We then present a case study illustrating the implications of direct coupling for neural network learning (Section 3). We show that the specific sensory surface of a robot can be exploited for learning. We then introduce examples of embedded networks—both hand-designed and evolved—that are able to exploit the robot's body to generate regularities in their input data (Section 4). Finally, based on the results obtained in the case studies we review by some of the key differences between embedded and traditional networks (Section 5).

2. Theoretical background

Embedded networks are neural networks, artificial or biological, that are connected to an agent's sensory and motor structures. The network receives its inputs from the agent's sensors, and its outputs result in specific movements of the agent. These movements, in turn, will lead to the subsequent input to the network. In other words, the network exists in a closed loop whereby the current sensory readings are determined by the network's previous output, at least in the quasi-static environments considered in this paper. Note that this situation significantly differs from the one faced by traditional networks. First, there is no distinction between a training and a test phase; the network must be able to learn the incoming patterns on-line. All information is local and enters the system, as the agent interacts with its environment. Second, a pattern's relevance must be determined; the network cannot just learn all incoming patterns since much of the input may originate from irrelevant parts of the environment or might simply be noise. This is a focus-of-attention problem. Moreover, the network must 'know' when to learn parts of the continuously changing input stream. Clearly, it would not be adequate to store all ever seen patterns in order to apply a full learning epoch for each new seen input pattern. The resulting computational complexity would be enormous. Third, an embedded network's outputs lead to its subsequent inputs; the network must be consistent with respect to its outputs. An error in the output will make the resulting input erroneous, which will in turn increase the probability of an even worse output. Errors may propagate 'through the loop' and make the agent's behavior meaningless. Thus, embedding a network in a complete agent that is capable of interacting with the world poses a number of challenges for the design of the network.

Salomon (1996) has recently summarized some key characteristics of an 'ideal' embedded neural network should have to meet these challenges:

(a) the network should feature incremental learning

properties due to the absence of a separate training and test phase; and
(b) the network should not suffer from problems caused by a too high VC dimension.

The effective VC dimension should be bounded by imposing constraints (Abu-Mustafa, 1992; Meier, 1997). The VC dimension is—very roughly speaking—a measure of the power of a learning machine (Vapnik and Chervonenkis, 1989). In what follows, we will focus on what these constraints are and how they can be taken advantage of, i.e., how they can be turned into opportunities. When designing embedded networks, we can in essence exploit the following constraints: (a) environmental constraints (Section 2.1), (b) morphological constraints (Section 2.2), (c) constraints resulting from an embedded network's capability to structure its own input through appropriate interactions with objects (Section 2.3), and (d) mobility constraints (Section 2.4).

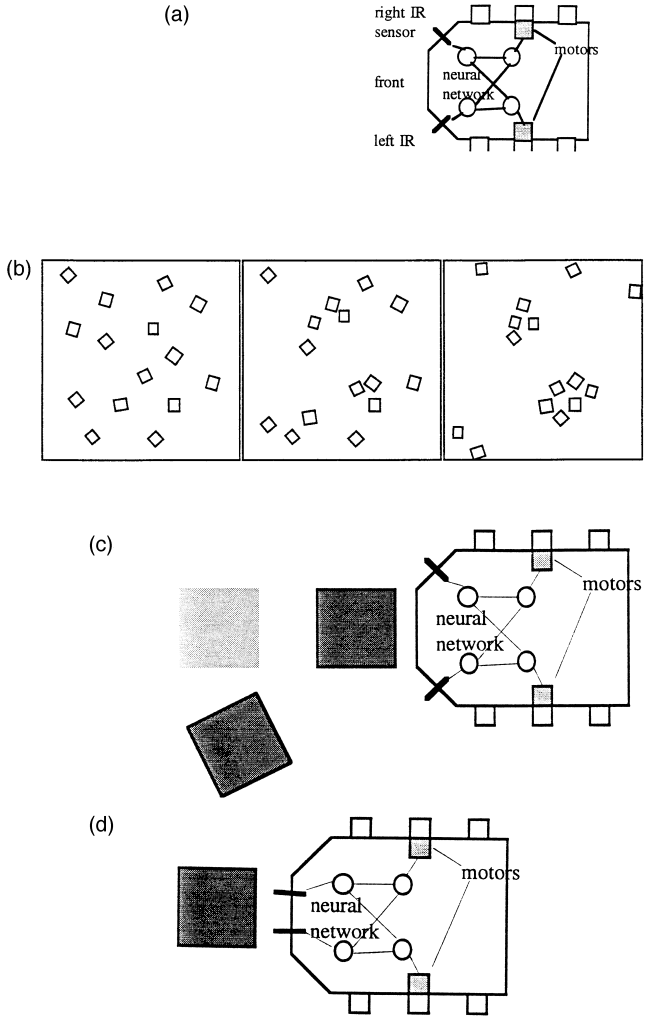


Fig. 1. Matching between brain and body. This figure illustrates how simple robots form clusters: (a) the robot with two IR sensors and a simple neural network that implements the avoidance reflex; (b) the clustering process (overall duration ca. 20 min); (c) explanation of cluster formation; (d) robot with changed positions of the sensors: its behavior will be entirely different even though the neural network is identical. For a more detailed account, see Maris and te Brockhorst (1996).

2.1. Environmental constraints

The definition of the ecological niche provides powerful constraints for the design of an embedded network. For example, in a world with symmetric objects, symmetry constraints can be built into the network (Abu-Mustafa, 1992). In a world with large objects (relative to the agent), continuity constraints can be built in. Another example is the robot built by Ian Horswill (Horswill, 1992, 1993). It is based on a vision system that exploits the fact that office floors are flat. A higher y-coordinate in the image implies that the object is further away (given that the object is standing on the ground). The constraints of the particular ecological niche (flat office floors, objects standing on the floor) significantly reduce the amount of information required to determine the relative distance of objects. This effectively means that the number of required parameters is reduced.

Such constraints reflect a priori assumptions about the data distributions in the environment (e.g. about the shape or size of objects). In the case of embedded networks, as we will show below, the data distributions are not given, but can be generated by specific interactions of the agent with the environment. But also learning problems that are intractable if considered from a purely computational view, often turn out to be benign if the constraints of a particular ecological niche are taken into account. Normally in the real world learning systems do not have to be universal, rather they are adapted to a particular niche. This can be illustrated using the example of the XOR problem. The learning theory tells us that this is a hard problem: it is not linearly separable (e.g. Hertz et al., 1991). Thus, powerful neural networks such as MLPs have to be used to learn this task. While XOR is an excellent case study to demonstrate the power of MLPs, it tells us relatively little about the real world. Fortunately, there is rarely a need to learn XOR. It has even been experimentally shown that natural systems (e.g. monkeys) perform poorly on XOR learning tasks (Thorpe and Imbert, 1989), and it is hard to think of natural situations in which the ability to solve an XOR problem would confer an advantage. On the contrary, if two cues both signal that food is about to arrive, when the two are present at the same time, the food is even more likely to appear (Thorpe and Imbert, 1989).

2.2. Morphological constraints

Embedded networks cannot process information that is not transmitted by the periphery, nor can they command behavior that is physically impossible for the periphery. As pointed out by Chiel and Beer (1997), properties of the periphery can simplify neural processing and control. For example, the human arm has a natural resting position: the relative stiffness of muscles around a joint creates an equilibrium point to which a limb will return after perturbation.

This is based purely on local processes at the joints of the arm, without central control. Similarly, by exploiting the dynamics of the periphery, walking behavior can be achieved that looks amazingly human-like but requires no internal processing (Garcia et al. (in press)). Another important constraint is the spatial location of the sensors. Sensors should be positioned so as to provide redundant information about the world. Such spatio–temporal correlations effectively impose constraints on the number of effective degrees of freedom of the input to the network. Moreover, these correlations can be exploited by simple learning mechanisms, as we will illustrate in case study I below (Section 3). They are also important in biological agents. A case in point is human development where it has been established that ''systematically correlated overlapping or concurrent information from more than one source is an important factor in behavior and development.'' (Millar, 1994, p. 40; see also Thelen and Smith, 1994). In adults, holding a small sphere between the forefinger and middle finger when these are crossed over each other generates the illusion that two objects are being touched (e.g. Millar, 1994). In other words, an illusion occurs when the usual covariance of different inputs is disrupted. The normal cause of covariation between forefinger and middle finger is, of course, due to morphological constraints, i.e., the fact that the two fingers are adjacent. In general, overlapping information can result from morphological constraints or from self-produced movement. The latter case is considered next.

2.3. Self-structuring of input

Mobile agents can take advantage of the fact that they can interact with their environment. The simplest examples are the correlated temporal changes in input patterns due to movement. In a recent robot model of visual processing, Almassy et al. (1998) have demonstrated that neurons showing translation invariance only developed when the robot was allowed to move; in their model, no translation invariance responses developed if the continuity of stimulus displacement was selectively destroyed, even if equivalent levels of sensory input were presented to the system. Moreover, the learning rule underlying this development was significantly simpler compared to other models where no embedding was used. For example, no explicit temporal delays were used to achieve translational invariant responses. Rather, the incoming patterns were naturally temporally correlated due to the agent's movements in space. We will discuss more complex ways of input-structuring in case study II below (Section 4).

2.4. Mobility constraints

One core problem in designing embedded networks is to estimate the computational power the network should have in order to solve the task. One important aspect of this concerns the dimensionality of the input data. In what follows we will use the term internal dimensionality to denote the dimensionality of the input data from an internal observer's point of view. In a sense, it is the dimensionality the neural network has to cope with. This internal dimensionality is analogous to the number of significant principal components (PCA) of the input data in the linear case. It is important to note that this dimensionality is different from the apparent degrees of freedom in the agent's body structure and sensor organization. Consider an agent equipped with eight proximity sensors and two motors. The maximum number of independently varying components in this agent is 8, i.e., it has eight sensor degrees of freedom (DOFs). Similarly, since the robot has two wheels whose speed can be independently controlled, the robot has 2 mobility DOFs (tangential speed and angular velocity). The important issue is that the following constraints act upon the sensor and mobility DOFs. First, in typical environments, objects are generally significantly larger than a single spacing between two proximity sensors. In such cases, at least two sensors covary due to the size of an object, and, consequently, the sensor data cannot have the full eight independently varying sensory components. Second, under the assumption that the environment is quasi-static, i.e., that objects around usually do not move, the number of independently varying components of the sensor data (principal components) collapses down to two. This is because the robot's movements are the only source of sensor data variation. This is a crucial point. It implies that—in embedded network design—one has to consider both the sensory *as well as* the motor degrees of freedom. Through the robot's actions both become coupled. Since the mobility in our example has two DOFs, the sensor data also has two DOFs. In other words, whatever arbitrary movement the robot generates, it experiences at most two DOF sensory patterns. Consider the case where the robot moves linearly back and forth against an object. In this case the sensory patterns reduce to one DOF. The main implications of these considerations are as follows. First, by carefully examining the number of sensor and mobility DOFs, one can arrive at an estimate of the dimensionality of the incoming sensor data. This in turn can be exploited in the design of the network. Second, the sensory DOFs are constrained by the mobility DOFs, i.e., by the motor system of the agent. Again, this can be used to estimate the size and required power of the network. Third, certain actions of the agent (e.g., linearly moving back and forth) further reduce the sensory DOFs, making the sensory data lower dimensional and hence easier to learn.

In sum, there are a number of constraints that can be exploited in embedded network design. The following case studies illustrate how this can be achieved in specific task environments and agents.

3. Case study I

In this first case study we develop a neural network

architecture, called 'Distributed Adaptive Control', for an autonomous agent which should learn to move about in an environment without hitting obstacles (Pfeifer and Verschure, 1992). The goal is to demonstrate how embodiment can result in morphological constraints that can be exploited for learning. Moreover, through self-structuring of its own input, this embedded network is able to select the relevant patterns it should learn, and adjust itself on-line and incrementally.

The ecological niche of the robot is a simple closed environment with randomly placed obstacles. The task of the robot is to learn to avoid the obstacles. A schematic representation of the set-up is shown in Fig. 2a.

In the case studies that follow the following generic scheme was used (Fig. 2b): the agents have a left–right symmetry, a number of infrared (IR) sensors, two wheels (with motors), and two sensors for measuring the speeds of the wheels. The IR sensors are used as proximity sensors yielding a measure of 'nearness' to an obstacle: the nearer, the higher the activation (the reading of the IR sensor). If the obstacle is far away, activation will be zero (except for noise). The IR sensors are also used as collision sensors: when they are saturated, this is counted as a collision (very high proximity). The wheels are individually driven by electrical motors. If both motors turn at equal speeds, the robot moves straight, if the right wheel is stopped and only the left one moves, the robot turns right, and if the left wheel moves forward and the right one backward with equal speed, the robot turns on the spot.

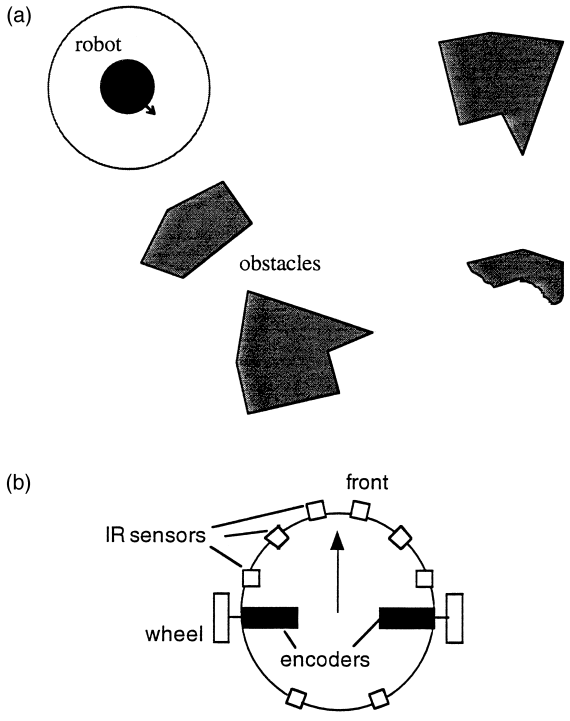


Fig. 2. Basic set-up for the experiments with 'Distributed Adaptive Control'. (a) Ecological niche. It is a closed environment with obstacles. The task of the robot is to learn to avoid the obstacles. The direction of motion of the robot is indicated with an arrow. The circle around the robot indicates the range of the proximity sensors (outside of this range, the sensors deliver no activation); (b) Schematic diagram of robot. The direction in which the robot is facing is indicated with an arrow. The IR sensors are used to implement proximity sensors and collision sensors (see text). The wheel encoders measure the speed at which the wheels turn. This generic scheme is used throughout the paper.

The robot has a number of built-in reflexes. If it hits with a collision sensor on the right, it will back up just a little bit and turn to the left (and vice versa). If there are no obstacles it will move forward. Fig. 3 shows how a neural network can be embedded in this robot.

Each sensor is connected to a node in the neural network: the collision sensors to nodes in the collision layer, the proximity sensors to nodes in the proximity layer. The collision nodes are binary threshold. Proximity nodes are continuous. Their value depends on the strength of the signal they get from the sensor. The proximity layer is fully connected to the collision layer in one direction. If there are six nodes in the proximity layer and six in the collision layer, as in Fig. 3, there are 36 connections between these two layers. The nodes in the collision layer are connected to a motor output layer. These connections implement the basic reflexes referred to earlier. The input to the node i in the collision layer is computed as follows:

$$h_i = c_i + \sum_{j=1}^{N} w_{ij} \cdot p_j \qquad (1)$$

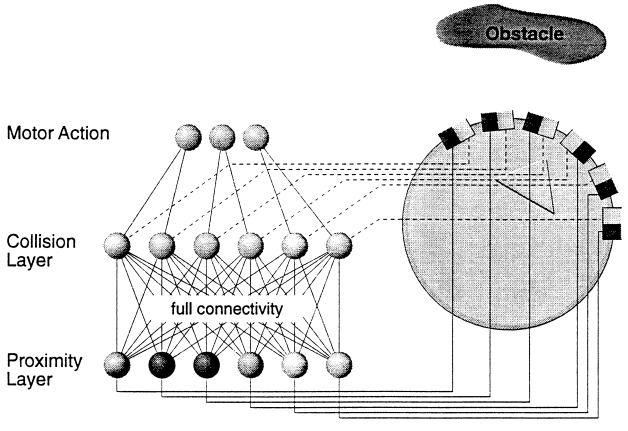


Fig. 3. Neural network embedded in the robot. There are three layers of nodes, a proximity layer, a collision layer, and a motor action layer. Proximity sensors are shown in black, collision sensors in grey. Each proximity sensor is connected to a node in the proximity layer (not all connections are shown). Similarly, each collision sensor is connected to a node in the collision layer. Proximity nodes have continuous activation levels and sigmoid activation functions, collision nodes are binary threshold. The proximity layer is fully connected in one direction (feedforward) by modifiable connections to the collision layer. The modification is achieved by a certain type of Hebbian learning. The collision layer is hard-wired to the motor action layer. This is to implement the basic reflexes which don't change over time. The gray levels in the proximity layer indicate the strength of the activation: the darker, the higher the activation.

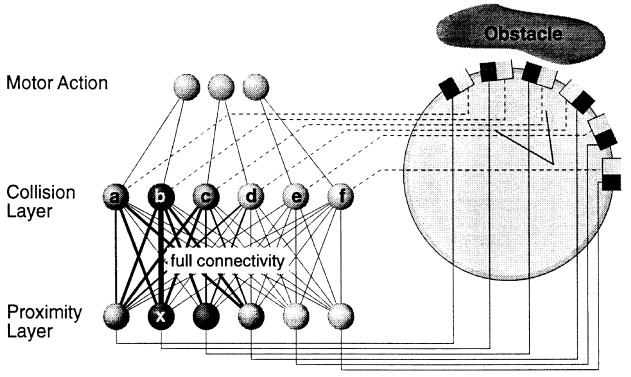


Fig. 4. The robot making a collision. Through the collision, the corresponding node in the collision layer is turned on. At the same time, there is an activation pattern in the proximity layer. The connections between the active nodes in the proximity layer and the active nodes in the collision layer are reinforced by Hebbian learning.

where p_j is the activation of node j in the proximity layer, c_i the activation resulting from the collision sensor, w_{ij} the weight between the proximity layer and the collision layer, and h_i the local field at collision node i. c_i is either 1 or 0, depending on whether there is a collision or not. p_j is a continuous value between 0 and 1, depending on the stimulation of the proximity sensor j (high stimulation entails a high value, and vice versa). N is the number of nodes in the proximity layer. Let us call a_i the activation of node i in the collision layer. Node i is—among others—responsible for the motor control of the agent. a_i is calculated from h_i by means of a threshold function g:

$$a_i = g(h_i) = \begin{cases} 0 : h_i < \Theta \\ 1 : h_1 \geq \Theta \end{cases} \tag{2}$$

The weight change is as follows:

$$\Delta w_{ij} = \tfrac{1}{N}\left(\eta \cdot a_i \cdot p_j - \varepsilon \cdot \bar{a} \cdot w_{ij}\right) \tag{3}$$

where η is the learning rate, N the number of units in the proximity layer as above, $\bar{a}$ the average activation in the collision layer, and ε the forgetting rate. Note that in the forgetting term we have $\bar{a}$, the average activation in the collision layer. This implies that forgetting only takes place when something is learned, i.e., when there is activation in the collision layer. The idea behind this active forgetting mechanism is that the current situation is slightly more important than the earlier ones. If the environment does not change, the weight matrix will converge to an equilibrium state. The factor $1/N$ is used to normalize the weight change.

When the robot hits an obstacle (Fig. 4) the corresponding node in the collision layer is turned on. As there now is activation in a collision node and simultaneously in several proximity nodes, the corresponding connections between the proximity nodes and the active collision node are strengthened through Hebbian learning. This means that next time around more activation from the proximity layer will be propagated to the collision layer. Assume now that the robot hits obstacles on the left several times. Every time it hits, the corresponding node in the collision layer becomes active and there is a pattern of activation in the proximity layer. The latter will be similar every time. Thus the same connections will be reinforced each time. Because the collision nodes are binary threshold, the activation originating from the proximity layer will at some point be strong enough to get the collision node above threshold without a collision. When this happens, the robot has learned to avoid obstacles.

We said earlier that in embedded networks there is no clear distinction between a learning and a test phase. Rather, the network has to be able to learn on-line and incrementally. Over time, the agent will start turning away from the object earlier. This is because two activation patterns in the proximity sensors, taken within a short time interval, are similar when the robot is moving towards an obstacle. Therefore, as the robot encounters more and more obstacles as it moves around it will continue to learn, even if it does no longer hit anything. This is reflected in the evolution of the weights between the collision and the proximity layer (Fig. 5).

Dark areas indicate large weights and light areas indicate small weights. At the beginning of the experiment (Fig. 5a), the weights are randomly distributed: there is no clear structure in the connectivity between the proximity and the collision layer. After the robot had collided 10 times this randomness has disappeared, and a structured connectivity pattern emerges (Fig. 5b). This pattern already makes the robot avoid walls. Learning does not stop even when the robot successfully avoids the walls. This can be seen in Fig. 5c: the connectivity pattern has now a clear diagonal structure reflecting the coactivations of the proximity and the collision nodes. This structure emerged because the collision and the proximity sensors are based on different physical processes, but are located next to each other on the robot's body, thus yielding correlated signals. For example, whenever the robot had a collision on the left, the proximity nodes on the left were highly active. Similarly, whenever the robot had a collision in the front, the corresponding collision and proximity sensors were activated. This correlation was then picked up by the Hebbian learning mechanism, resulting in the diagonal weight pattern shown in Fig. 5c.

Note that the correlation implies that there is a certain amount of redundancy: the information the network gets from one sensor overlaps to a certain degree with the information it gets from the other. This overlap, i.e., the correlations between proximity and collision sensors, is the basis for learning in the network. More generally, spatio–temporal correlations across different modalities are constraints that can be exploited for learning in embedded networks. The main idea is illustrated in Fig. 6.

In the architecture discussed here, whenever there is a collision, there is high activation in the proximity sensors that are adjacent to the particular collision sensor. Thus, if

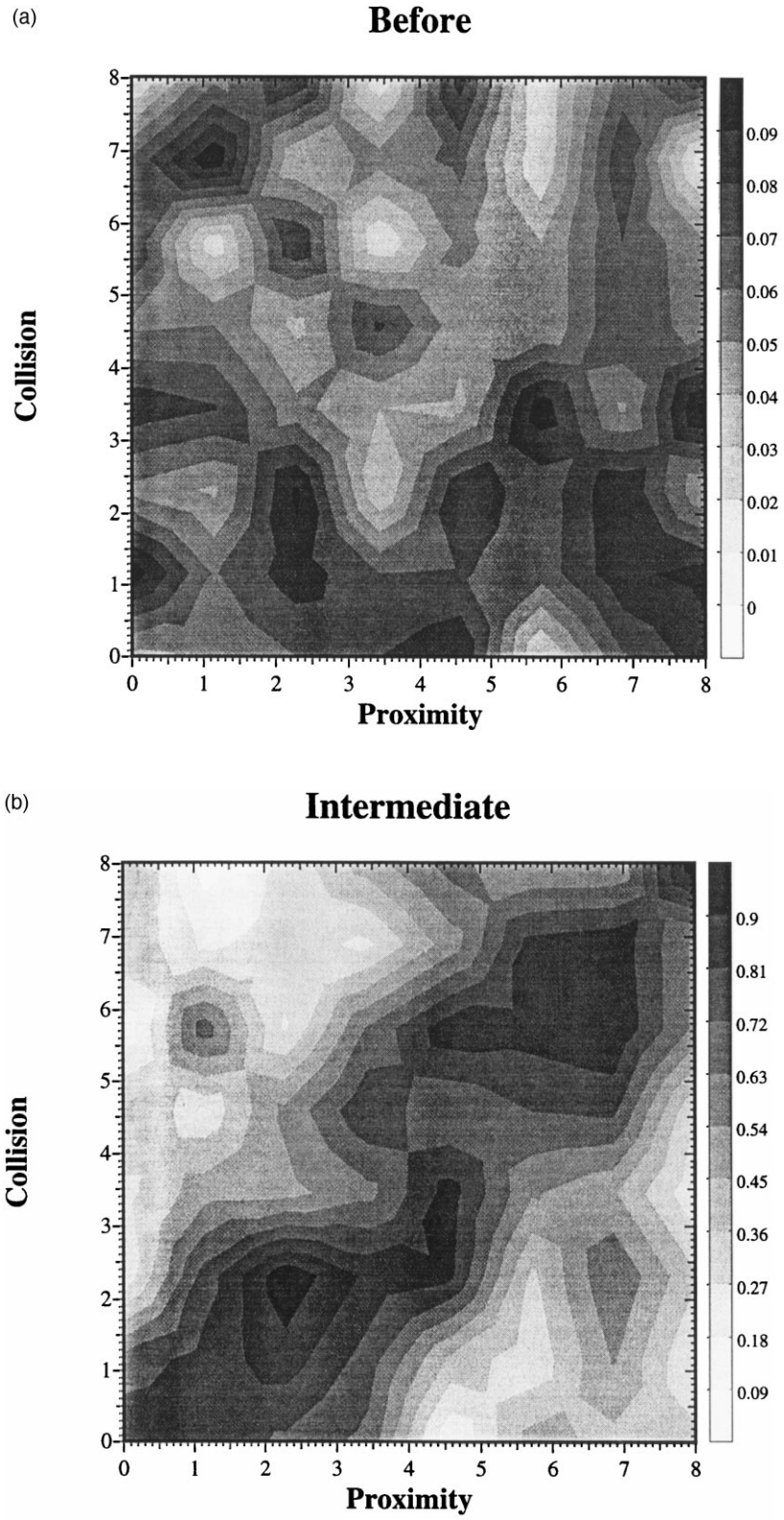


Fig. 5a,b. Evolution of weight patterns between proximity and collision layers. Dark areas indicate strong weights, light areas indicate weak weights. This example was recorded from a robot with eight collision and proximity sensors. (a) Weight pattern at the beginning of the trial. There is no apparent structure, the weights are randomly distributed; (b) weight pattern after the robot had collided 10 times. A diagonal pattern emerges. (c) see next page.

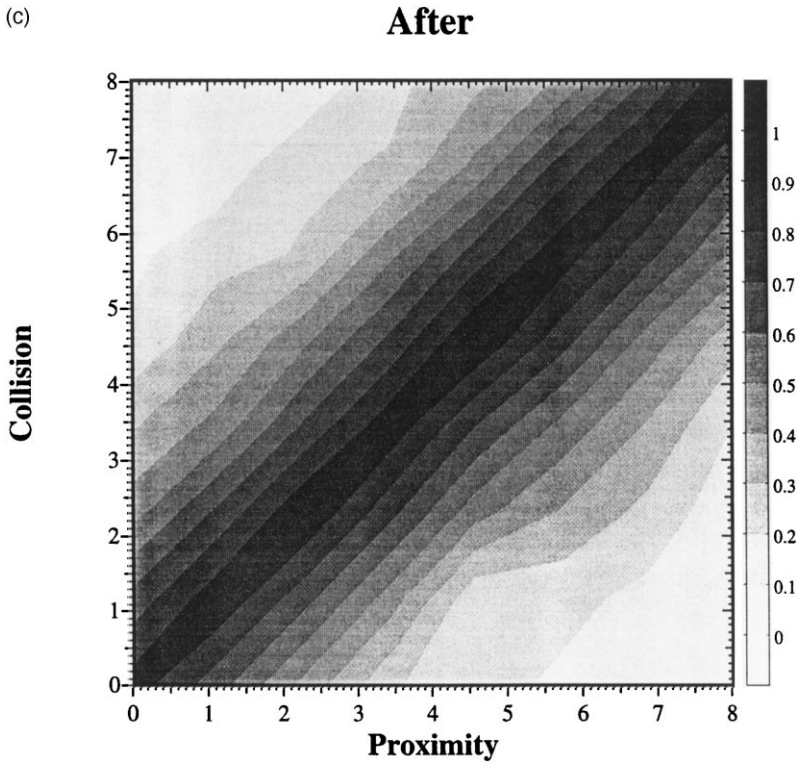


Fig. 5c. Evolution of weight patterns between proximity and collision layers. (c) weight pattern after the robot had encountered (not necessarily collided) a wall for another 10 times. The diagonal structure is now very clear and robust. The robot does not collide with walls any longer.

there is a collision, the information from the IR sensors is largely redundant. In other words, the agent is getting information twice, once over every channel. Clearly, this only works if the sensors are positioned appropriately. This can immediately be seen by inspecting the robot in Fig. 6b. If this robot hits an obstacle, there is no overlap in the spatial information delivered by the two sensory channels and there is nothing to be learned. But the robot can behave in the real world. For example, it could simply continuously move back and forth, such that both types of sensors are in the front of the robot alternately. Near an obstacle the sensory signals from the different channels will be correlated, just as for the properly designed robot in Fig. 6a. This type of behavior belongs to the class of mechanisms for self-structuring of input which we discuss in more detail in the second case study below (Section 4).

This shows that the design decision of where to put the sensors becomes highly involved: the potential overlaps in various types of sensory–motor coordinations have to be taken into account. What kinds of sensory–motor

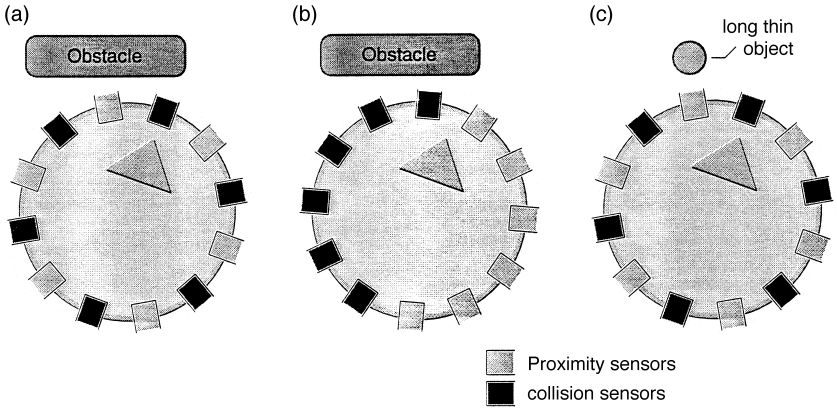


Fig. 6. Illustration of the concept of spatial information overlap: (a) Proximity and collision sensors are mixed. This agent is designed to yield spatial information overlap; (b) all proximity sensors are on one side of the robot whereas all collision sensors are on the other. In this robot there is no spatial overlap in the information from different sensory channels. Of course, nobody would design a robot like the one in (b): it is a caricature to illustrate the point. (c) Same robot as in (a), but the environment consists only of long thin objects.

coordinations are possible or necessary depends, in turn, on the environment. Imagine for a moment, that the robot in Fig. 6a operates in an environment with only narrow, vertical objects (Fig. 6c). Because the objects are not large enough to stimulate both sensors simultaneously there will be no correlations. The robot in Fig. 6a would have to engage in some kind of behavior in order to generate the appropriate redundancy required for learning.

In conclusion, the network is able to learn the task because of the constraints that exist in its sensory input. These constraints are due to the spatial location of the sensors, rather than to some internal process. This illustrates how morphological constraints can be exploited for learning. Moreover, the continuity of incoming sensory patterns is a prerequisite for the learning process to be successful. This is a simple form of self-structuring of input. The next case study (Section 4) illustrates more complex ways of how mobile agents, through embedded networks, can structure their input, thereby generating appropriate constraints for learning.

4. Case study II

4.1. Two learning problems and geometric separability

The task chosen in the following case study is categorization. In most experiments the agent is required to learn to distinguish between different (usually two or three) categories. That this task is indeed far from trivial to solve when considering real-world objects is illustrated by a study by Nolfi (1996). Nolfi has studied a Khepera™[1] robot—which we will also use in the experiments presented below—whose task was to distinguish between walls and target objects (small cylinders). The walls and target objects were sampled by placing the robot in front of them and storing the activations of the eight IR sensors for 180 different orientations and for 20 different distances. These data are comparable with those used in the case studies presented here as they are originating from the same robot faced with similar objects to the ones used below. They were then used to train a backpropagation network to categorize the two types of objects.

The problem seems to be simple because walls are very distinct from the small target objects used in the study. The results of these experiments were as follows. Networks with no hidden units correctly categorized 22% of the patterns, while networks with hidden units, on average, were correct in 35% of the cases. Adding an additional four hidden units did not improve performance significantly. Thus, these networks showed a very poor categorization performance. Obviously, the problem is not one of lack of computational power. Rather, the main reason for the networks' poor performance is the ambiguity in the sensory data. Sensory patterns could only be correctly categorized in a rather narrow range of angles and distances. While to an outside observer it might seem surprising that the networks have such difficulties distinguishing between two so obviously different categories as cylinders and walls, it is less surprising if we look closely at the robot itself. Note that—apparently—to the robot, because of its shape, the spatial location of the sensors, and the physical properties of the sensors (e.g., short range) the two are not as different as it might seem to us. As we will show below, sensory–motor coordination, i.e., mechanisms that enable the agent to structure is input, maxi-mizes the differences between the two.

The problem is to be seen in the general context of the problem of object constancy, i.e., the fact that one and the same object can lead to a very large number of input patterns depending on the distance, the viewing angle relative to the object, the lighting conditions, noise in the sensors etc. Fig. 7 shows the activations of the two front IR sensors for two objects at 11 adjacent angles and distances.

The sensor readings vary significantly as the robot changes direction and distance relative to one and the same object. Another problem involved in real world category learning is that data for two categories (small and large cylinders in this case) can have a significant overlap. This can also be seen in Fig. 7. The IR sensor readings for the two categories used in the experiments presented below are highly correlated both in time and (sensor) space.

Clearly, these problems make the task of (category) learning in the real-world a highly non-trivial one. The main problem is that regions of input space that correspond to one and the same object are not located nearby, but are scattered throughout the whole sensor space. Moreover, regions corresponding to different objects may be located next to each other or may even be overlapping. Put differently, the distance in sensor space for data originating from one and the same object can be large, while the distance between two objects from different categories can be small. This is a problem for most neural networks because they, in essence, aim to divide the input-space into regions such that all the points in a given region map onto the same output-space point (e.g., category). The underlying assumption is, of course, that the function to be learned is smooth, and that therefore input points belonging to the same category (output) will tend to cluster together in the same region of input space. As the two problems just mentioned illustrate, however, this assumption is not valid in a significant number of tasks (see also Thornton, 1997). Clearly, however, there must be *some* structure in the data, otherwise learning would not make sense in the first place.

In a recent paper, Clark and Thornton (1997) have introduced the concept of type-2 problems to denote high-dimensional

[1] Khepera™ is a circular miniature robot with about 5 cm diameter and is, depending on how it is equipped, 5 to 10 cm high. Its infrared (IR) sensors have a maximum range of about 5 cm

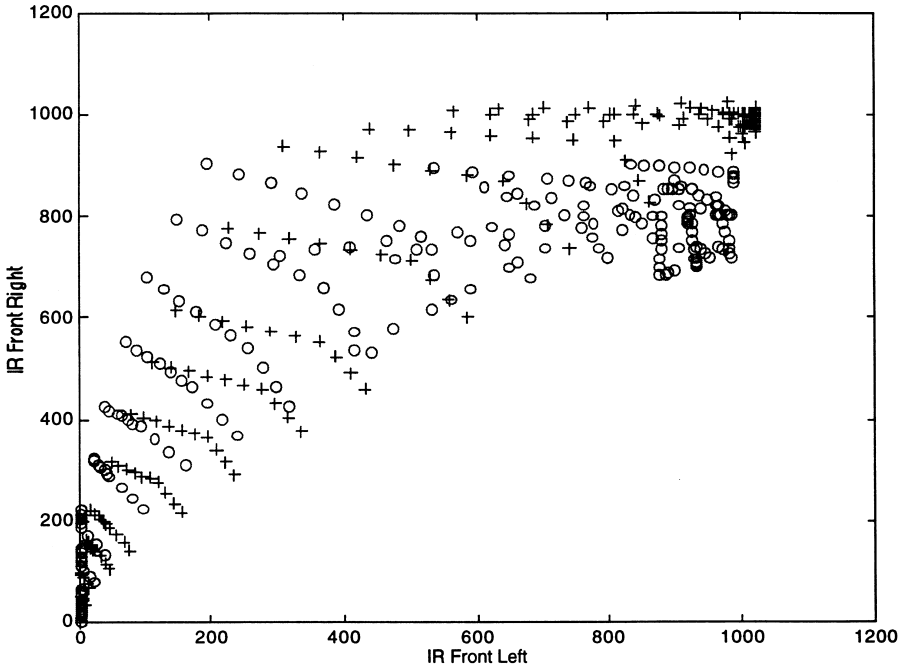


Fig. 7. IR sensor readings for small and large objects. The data were taken from the front two sensors of the robot. The robot approached a small and a large object. The data are not only noisy, but there is significant overlap between the data from the two types of objects. Circles indicate activities of the IR sensors for small objects, plusses activations for large objects.

spaces in which regularities appear only 'hidden' or are only 'marginal'. They distinguish type-2 (intractable) problems from type-1 problems where the regularities are apparent in the input data. Type-1 problems can be learned by an appropriate learning mechanism, i.e., one that is able to pick up regularities in the input space (e.g., backpropagation or even simple Hebbian learning). This is, however, not the case for type-2 problems where these regularities are not obvious and can only be recovered by means of appropriate transformations—'recodings' as Clark and Thornton call it—of the input data. In this case, type-2 problems become type-1 problems and can be learned.

For example, if we take the data shown in Table 1, the conditional probability that the output is 1, given that $x1$ is 1, is 0.5, or the conditional probability that the output is 1, given that the input $x2$ is 2, is 0.67 (see Table 1, left). Most of the conditional probabilities are close to chance. However if the data are transformed, i.e. if we calculate $x3$ as the difference between the values of $x1$ and $x2$ the conditional probabilities are 1 (Table 1, right). For example, the conditional probability $p(y1 = 1|x3 = 1) = 1$. If we interpret $x1$, and $x2$ as sensory stimuli and $y1$ as a category representation, we can say that in the first case, the mapping is not visible in the data (Table 1, left) whereas in the transformed case it is easily discernible. Note that the fact that a data set is not of type 1 does not automatically imply that it is of type 2—there may be no regularities in the data set whatsoever, not even 'hidden'.

We suggest that the main problem in (category) learning about real-world objects is to turn type-2 problems into type-1 problems. The above mentioned study by Nolfi (1996) suggests that even a universal learning device such as a backpropagation network cannot learn the distinction between walls and small objects from raw data only. This indicates that additional mechanisms are needed that impose certain constraints on the sensory stimulation. Put differently, the main problem lies in the fact that real world data such as those collected in Nolfi's study constitute type-2 learning problems and hence first have to be transformed (or they may not be learnable in the first place, i.e. they may not even be of type 2).

Table 1
Illustration of type-1 and type-2 problems. Adapted from Clark and Thornton (1997)

$x1$	$x2$		$y1$	$x3$		$y1$
1	2	⇒	1	1	⇒	1
2	2	⇒	0	0	⇒	0
3	2	⇒	1	1	⇒	1
3	1	⇒	0	2	⇒	0
2	1	⇒	1	1	⇒	1
1	1	⇒	0	0	⇒	0

There are two main strategies how this can be achieved. First, the internal processing of the input can be improved. Clark and Thornton (1997), for example, suggest that already learned representations can be used to reduce the complexity of learning subsequent patterns. In other words, the early knowledge is a kind of 'lens' through which input data are recoded. (Clark and Thornton, 1997, p. 63). The idea is that instead of learning the whole input space at once, the problem can be broken down and made more tractable by 'starting small', i.e., by learning some basic properties of the input, and then use the already learned knowledge to 'filter' the subsequent input. Another possibility to increase the power of internal processing is to incorporate simple constraints such as symmetry in the network structure (see our discussion of constraints above).

The second approach to transform type-2 into type-1 problems is directly derived from the basic tenets of embodied cognitive science. It consists in exploiting processes of system–environment interaction. In other words, the idea is to exploit the fact that a mobile agent can actively structure its input by manipulating the world. We call this way of interacting with the world sensory–motor coordination (e.g. Pfeifer and Scheier, 1997; Pfeifer and Scheier, in press-a; Scheier and Pfeifer, 1995). We suggest that this manipulation can serve to transform a problem that would be of type-2 (without sensory–motor coordination) into one of type-1 through appropriate mechanisms of self-selection of input data.

The notion of type-1 and type-2 data sets can be quantified by means of the geometric separability index (*GSI*) introduced by Thornton (1997). Geometric separability is a generalization of linear separability. The *GSI* measures how close regions in sensor space belonging to the same object are to each other. It is defined, for a given task, to be the proportion of nearest-neighbor inputs with identical output (e.g., category):

$$GSI(f) = \frac{\sum_{i=1}^{N} f(x_i) + f(x_i') + 1 \bmod 2}{N} \tag{4}$$

where f is a binary target function (the category to be learned), x is the data set consisting of N vectors, and x_i' is the nearest neighbor of x_i. The *GSI* has some properties which make it an interesting tool for further investigating the differences between type-1 and type-2 learning problems. Thornton (1997) has shown that the *GSI* can predict with high accuracy the performance of even the most sophisticated learning algorithms by simply computing the geometric separability of the data set. The important point here is that type-2 data sets have low *GSI* values, while type-1 data sets have large *GSI* values (Thornton, 1997). Note that the reverse is not true: from the fact that the *GSI* is low, we cannot conclude that the data are of type-2. For example, for the XOR (or the parity problem in general) the *GSI* is 0. However XOR and parity in general can be learned by an MLP. But clearly, low *GSIs* make for hard learning tasks.

We computed the *GSI* for the data set consisting of the IR sensor readings taken at 31 distances (from just visible to right in front of the object) at 200 different angles. Fig. 8 shows the *GSI* for all 200 angles. The mean *GSI* for these data is 0.47 ($\pm$ 0.11) indicating that these data constitute a hard learning problem. They may in fact be of type-2, but we cannot tell for sure by simply calculating the *GSI*. However, the study of Nolfi (1996) referred to earlier suggests that the data are indeed of type-2. In other words, the agents have to be equipped with additional ways to interact with the obejcts so as to transform the type-2 data into type-1 data (to icnrease the *GSI*). A better way of saying this would be ''to generate type-1 data'', rather that to ''transform'' the data (because the data are not given and then transformed). This, in essence, involves embedding the network in an agent and allowing it to interact with the objects whose categorization it is supposed to learn.

4.2. Designed sensory–motor coordination

In the following experiment, a task similar to the one used in the Nolfi experiments was used: the agent has to learn to collect some types of objects (e.g., small ones) while ignoring others (e.g., large ones), i.e., it must be able to categorize the objects in its environment. The main goal was to design the agent such that it would be able to learn the categories based on mechanisms of sensory–motor coordination. We call this type of agent SMC (for *s*ensory–*m*otor *c*oordination) agent. We focus on SMC I, the first in a series of agents instantiating the general idea of sensory–motor coordination (see Scheier and Pfeifer, 1995 Scheier and Lambrinos, 1996 Pfeifer and Scheier, 1997 Pfeifer and Scheier, in press-a). The ecological niche is a flat environment with a home base to which the robot had to bring the objects it collected (Fig. 9a).

Two types of objects were used: small and large cylinders with a height of 3 cm and 2 cm, and a diameter of 1.5 cm and 4 cm, respectively. The task of the agent is to learn to bring the small ones to a home base while avoiding the large ones. In all experiments, there were 15 objects of each category randomly distributed over the whole arena. The robot is shown in Fig. 9b. The arm of the gripper can move through any angle from vertical to horizontal, whereas the gripper can assume only an open or a closed position. Arm and gripper positions can be sensed by position sensors coupled with the respective motors. The arm-position sensor takes values from 9 (bottom back) to 255 (bottom forward). Objects inside the gripper can be detected by an optical barrier that is mounted on the gripper.

The sensory system of the SMC I agent was identical to the one used by Nolfi. From his analysis we know that through this sensory system the objects cannot be learned by trying to identify a mapping from sensory stimulation

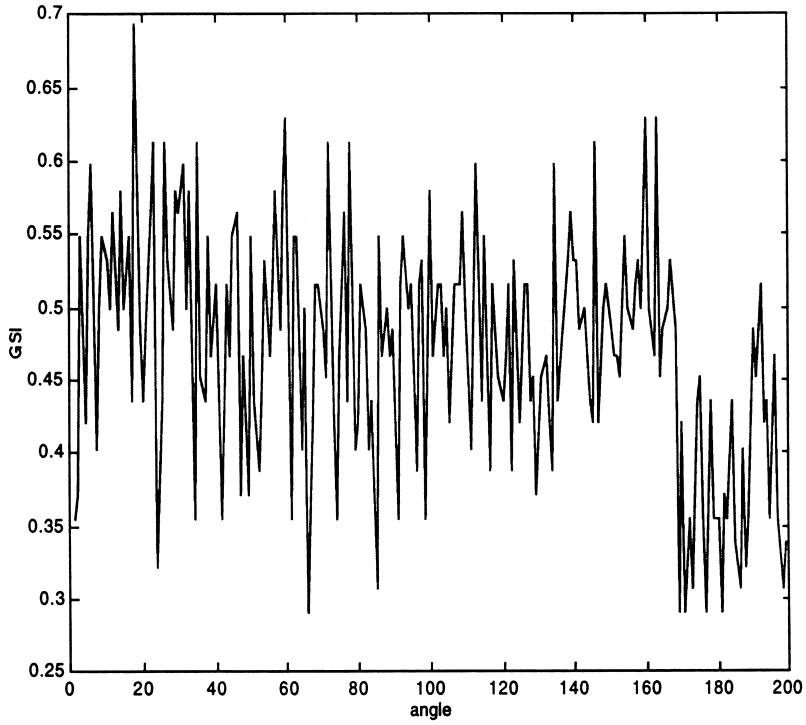


Fig. 8. Geometric separability index (*GSI*) for real-world categories. Sensor data from large and small cylinders recorded at 31 distances and 200 angles. At angle 0, the objects are 90 degrees to the right of the robot, at angle 90, the objects are in front of the robot.

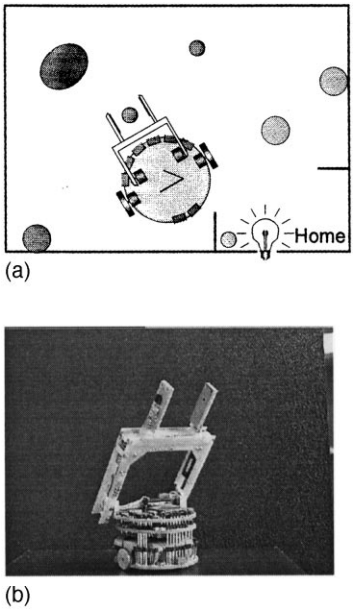


Fig. 9. Set-up for experiment with SMC I agent. (a) The ecological niche of the SMC I agent. There are small and large objects. The task of the agent is to bring the small objects to a predefined location ('Home'), and to learn to avoid exploring the large objects. (b) The Khepera™ robot with an arm-gripper system used in the SMC I experiments. The arm of the gripper can move through any angle from vertical to horizontal, whereas the gripper can assume only an open or closed position. Arm and gripper positions can be sensed with position sensors mounted in the respective motors. In addition, there is an optical barrier inside the gripper with which the presence of objects can be sensed.

onto internal representation. The SMC I agent is equipped with eight IR sensors. Each IR sensor has 2^{10} (or roughly 1000) states which amounts to an input space consisting of roughly 10^{24} different states. The categorization task is further complicated by the fact that there is a significant amount of noise in the sensor data, and that the sensor readings for the two types of objects overlap (see Fig. 7 above).

The main idea of our approach is to reduce this high-dimensional space by appropriate interactions with the object by imposing some structure or constraints on the number of states the various sensors can occupy. One way to do this is to generate spatio–temporal correlations in the sensor space by interacting appropriately with the object. In the case of the SMC I agent, the sensory–motor coordination leading to a reduction in sensor space is *circling*. This circling behavior induces high spatio–temporal correlations in the sensor patterns (see below). It is equivalent to the object rotation behavior found in human infants (Ruff, 1984).

Circling is achieved by three simple networks which were continuously and asynchronously active. The effect is as follows: Normally, the agent moved forward (`move-forward` network) and when encountering an obstacle, it avoided it by turning away (`avoid-obstacle` network). At the same time, if there is stimulation in one of its lateral (left or right) sensors, it turned slightly towards the object (`turn-towards-object` network). The interaction of these three networks leads to the behavior shown in Fig. 10.

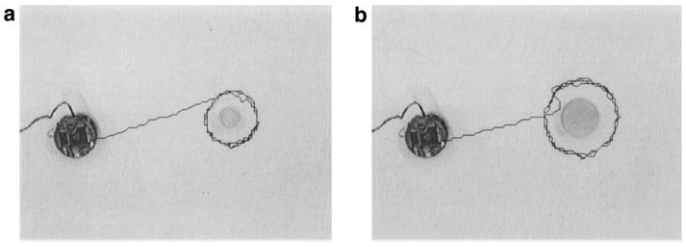


Fig. 10. (a) SMC I agent circling around small object; (b) agent circling around large object.

This is a form of sensory–motor coordination: sensors and motor actions are coupled via the basic processes. For category learning, the motor speeds were used in addition to these sensor activations. The important point about these motor speeds is that they are different for the two types of objects. We can compute the angular velocity of the robot by subtracting the motor speed of the left motor from that of the right motor. This angular velocity is larger for small objects than for large ones. This is shown in Fig. 11.

In this example, the robot first moved in the open plane for about 40 steps. It then started to circle around a small object for about 80 steps. After it had left the object, the robot first moved in the open plane again, then avoided a large object (indicated by large fluctuations around 150 and 170 steps), and finally started again to circle around a small object at around 180 steps. Note that the absolute value of the angular velocities are different for the two types of objects. This difference can be used for category learning. In addition, however, we want the learning to be based on the sensory readings. We said at the beginning of this section that this learning task is hard because of the high-dimensional state space. It turns out that the circling behavior just described, an example of sensory–motor coordination, significantly reduces this space.

For the following analyses we had the robot move across the open plane, approach an object and circle around it. We recorded the sensory space of the robot by means of 10-dimensional vectors consisting of the current readings from the eight IR sensors and the two motor speeds. The process of sensory–motor coordination—circling in this case—should lead to high correlations in these data. Two kinds of correlations were calculated. The first is the correlation between the sensor readings over time: the 10-dimensional vectors of the sensor readings are taken over a time window of 10 steps and the correlations calculated. If the pattern is stable over time, the correlation will be high, if there is a lot of change, it will be low. This is shown in Fig. 12.

The second correlation is the one between the different sensor activations: this correlation will be high if the covariation between the sensors is high. In other words, the correlation can be high even if there is a lot of change in the sensor readings. Of course, if the pattern is stable over time—which is the case when the agent is circling around a cylinder—the sensor readings will also be correlated.

If we inspect Fig. 12 we find that the correlation is at an intermediate level as the agent moves about in the open (due to noise—if there were no noise, the correlation would be maximal). As the agent approaches an object, the correlation drops because now there is rapid change in sensory activation. Once the agent is near the object, the dynamics of the reflexes begins to play and we have—plus minus

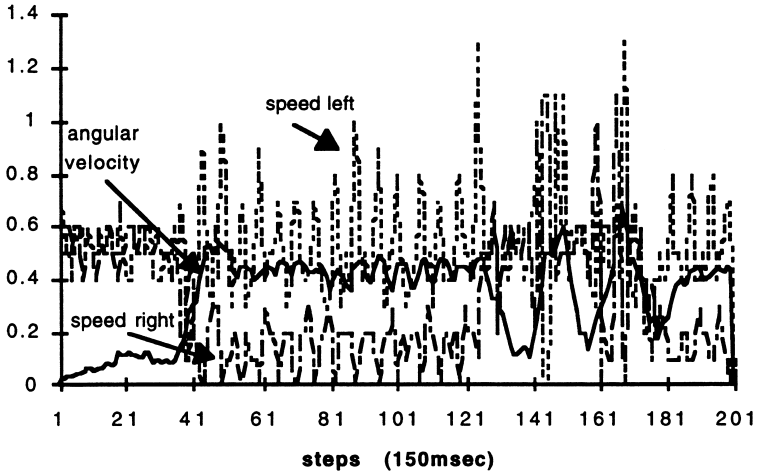


Fig. 11. Motor speeds and resulting angular velocities of the circling behavior. The robot first moved in the open plane for about 40 steps. It then started to circle around a small object for about 80 steps. After it had left the object, the robot first moved in the open plane again, then avoided a large object (indicated by large fluctuations around 150 and 170 steps), and finally started again to circle around a small object at around 180 steps.

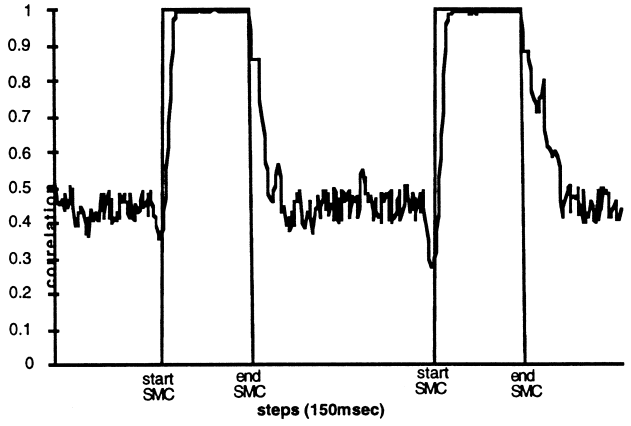


Fig. 12. Change in correlations between subsequent input vectors as the agent is approaching and then circling around a large object. Vectors are 10-dimensional: eight IR sensors and the two motor speeds. 'Start SMC' means that the agent starts circling around the object (analogously for 'end SMC'). The arrow indicates the drop in correlation as the agent approaches an object.

noise—time-locked activity in the sensory space. This can be seen in the correlations which rapidly jump to the maximum level. Note that these correlations are induced by the agent's own movements or, in other words, by the sensory–motor coordination. And the sensory–motor coordination requires embodiment. Stated differently, through its own behavior, the agent generates *redundancy* in the sensory signals. And, as is well-known, redundancy is the prerequisite for learning (Ashby, 1956). We have shown previously, that the sensor–motor coupling leads to a reduction of the degrees of freedom in the input space. More specifically, the different objects can be learned based on one single dimension (that can be mapped onto angular velocity) (Pfeifer and Scheier, 1997).

The fact that there are correlations, i.e. that there is a stable pattern over time, also yields a focus of attention mechanism: whenever there are correlations, it is a good idea not only to heed the current situation, but also to learn. Because the stream is no longer just changing in arbitrary ways, there are very clear correlations that turn the category learning problem into a tractable (type 1) one.

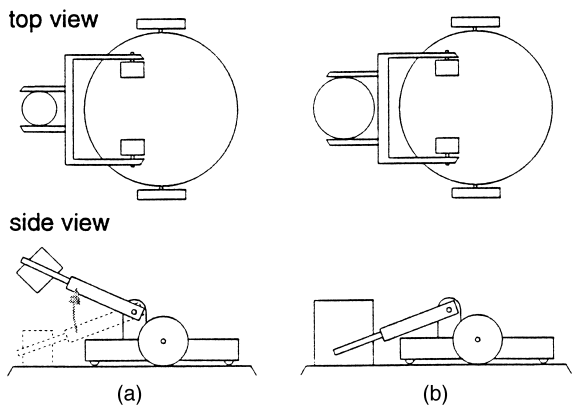


Fig. 13. Different gripper positions lead to different behaviors. (a) if the object can be lifted, the robot brings it to the home base; (b) if the object is too heavy to be lifted, the robot turns away.

As an aside, note that the correlations shown in Fig. 12, although they depend on the robot's individual characteristics, are 'objective' and can be numerically calculated. Therefore they can, in principle, be picked up and exploited by the agent itself, thus providing the basis for what one might call 'self-cognition'. The agent can acquire a notion of its own emergent behavior. In addition to high correlations, sensory–motor coordination should result in a high separability between categories. This is indeed the case: the mean *GSI* resulting from the circling behavior is 0.95 ($\pm$ 0.04). For a more detailed quantitative analysis of this reduction in terms of principal components and Bartlett's dimension, see Pfeifer and Scheier (in press-b).

After the agent has moved along an object for some time, i.e., after it has had continuous stimulation of a lateral sensor, a reflex was triggered which enabled the agent to find out what it could do with the object. This consisted in lowering the gripper over the object (Fig. 13). The robot first tried to grasp all objects. If the object was graspable and the agent could pick it up, a value signal was generated which reinforced the association between the sensory–motor sequence the agent had engaged in right before grasping and the grasp process.

If the agent had successfully grasped an object (Fig. 13(a)), it would bring the object to a home station. If the object was too heavy, it could be picked up and the agent would turn away from the object (Fig. 13b). In order to be able to reinforce the connections between a particular sensory–motor coordination and the networks for the different processes the sensor readings from the IR sensors and the wheel encoders were projected into a Temporal Kohonen map (Chappell and Taylor, 1993) with leaky integrators for all the sensor variables. This provides a certain amount of information about the agent's recent sensory–motor past. The important point is that the Kohonen map can pick up the regularities induced by sensory–motor coordination. The learning converges very rapidly, on average after the robot had encountered about 10 objects. Table 2 summarizes the results of the experiments.

The numerical results show that the number of steps before the agent engages in the right sort of behavior is significantly reduced after the agent has encountered 10 objects.

4.3. Evolved sensory–motor coordination

The agent architecture presented in the previous section

Table 2
Number of steps along object before engaging in appropriate behavior, i.e. picking up or turning away (averaged over 50 trials)

SMC I		
Object type	Initial	After 10 objects
Small	40.2 $\pm$ 1.44	14.2 $\pm$ 1.76
Large	41 $\pm$ 2.11	14.4 $\pm$ 2.08

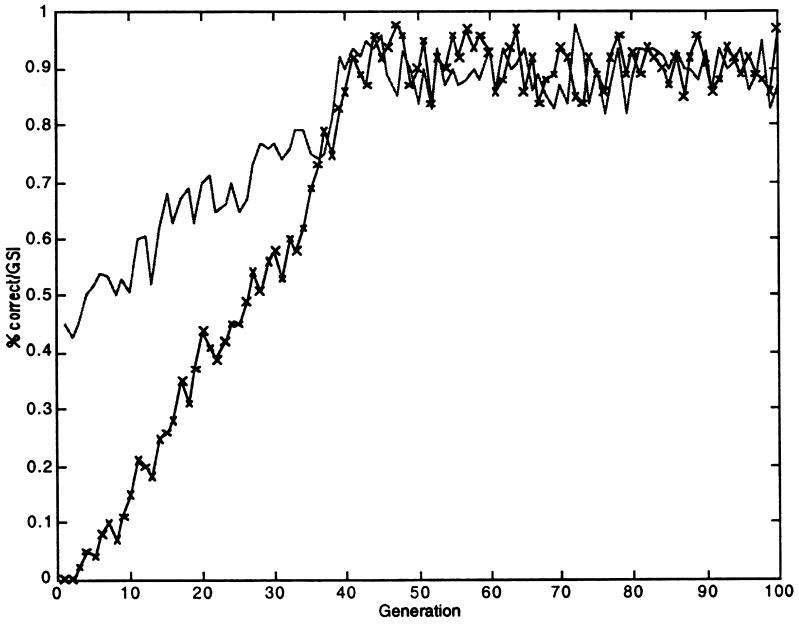


Fig. 14. Results for evolved sensory–motor coordination. The line with crosses shows the mean percentage correct of the best individuals, averaged over the four network types (recurrent, and feedforward, with 0 or 4 hidden units, respectively). The line without crosses shows the corresponding *GSI* value. The data are means over 30 runs.

was hand-designed. In order to further test the idea that mechanisms of sensory–motor coordination can be used to solve the category learning problem we have applied methods from evolutionary robotics to evolve rather than hand-design the control architecture for agents facing the same category learning task. More specifically, the agents had to learn to distinguish between small and large cylinders by avoiding small cylinders, and staying close to large ones. The agents in this case were simulated, but the sensory readings were taken from a look-up table containing the sensory readings of the real robot shown in Fig. 2b.

Four types of neural controllers were evolved: feed-forward and recurrent networks with eight and 12 neurons each. Out of these neurons, six were receiving input from the six IR sensors, and two were connected to the motors of the robot. The process started with 50 randomly generated genotypes each representing an embedded network with the architecture just described, and a different set of randomly assigned connection weight. Weights were represented by a 6-bit binary string, where the first bit determined whether the connection was to be used or not, and the remaining bits coded the value of the weight in the range -1 to 1. Moreover, for each neuron a threshold value was encoded with six bits. Thus, the total length of the chromosome for a network with 12 neurons was 936 bits (12*12*6 bits + 6*12 bits). Individuals were allowed to 'live' for five epochs, with each epoch consisting of 5000 actions. At the beginning of each epoch, the agent and the objects (four of each category) were randomly positioned in the environment. Fitness was incremented whenever the agent was close to a large object, and decremented whenever the agent was close to a small object or a wall. The ten best individuals of each generation were allowed to reproduce by generating five copies of their embedded network. Each copy contained random mutations which affected the resulting connection weights. Mutations were obtained by replacing 5% of randomly selected bits with a new randomly chosen value. The population size was 50, and in a typical run 100 generations were evolved. In each generation, 50 genes (bits in chromosome) are randomly changed. The encoding was a direct one-to-one mapping in which the phenotype is directly encoded in the genotype. In total, the evolutionary process was run 30 times.

Fig. 14 shows the mean ($N = 30$) percent correct of the fittest individual of each generation, averaged over all evolved network types, together with the resulting *GSI* of the input data. On average, the evolutionary process stabilized after about 40 generations, meaning that the agents were able to make reliable distinctions between the categories.

This is reflected in the *GSI* values that saturate after about 40 generations. This effectively means that, indeed, these agents solved the categorization task by structuring their input, rather than using internal transformations of the input data. In other words, in contrast to the (isolated) backpropagation network, these embedded networks could learn the distinction between the two categories.

The fittest individuals in 86% of the 30 runs used some sort of self-structuring to solve the categorization task, i.e., the motor speeds were non-zero at all times. The remaining 14% approached the object, and then stopped in front of the object. The performance of these latter agents

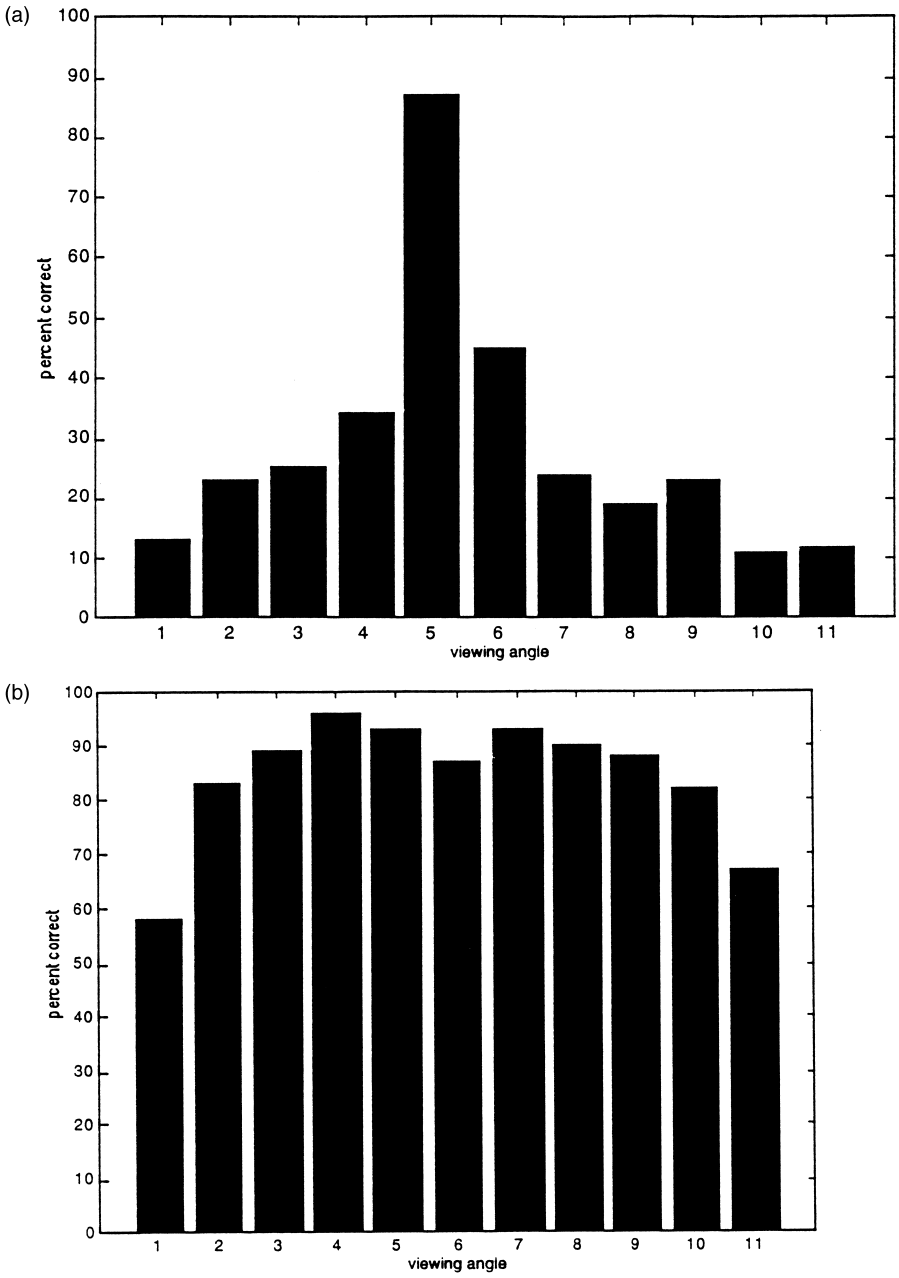


Fig. 15. Performance of evolved controllers: (a) agents that stopped in front of objects; (b) agents that engaged in the circling sensory–motor coordination. Performance was computed by letting the agents approach a large object from 11 different angles. Data show means taken from 20 runs for each angle.

was, however, significantly less stable. They typically were able to categorize the object only from certain view points (Fig. 15).

From the theoretical perspective presented earlier, it follows that the latter agents were not able to appropriately structure their own input, and were thus confronted with data of type 2. In other words, it seems likely that the more successful agents were imposing constraints on the sensory data in order to solve the categorization task. The main mechanism they used was the one of circling which evolved in 69% of the runs. In essence, two types of circling behaviors were observed (Fig. 16).

First, agents that were controlled by feedforward nets always stayed close to the object. Second, agents with recurrent nets made larger circles or ellipses, sometimes loosing direct sensor contact with the object. In other words, the evolutionary process came up with a mechanism of sensory–motor coordination that was essentially the same as the one designed by the authors to learn real-world categories. The main function of this

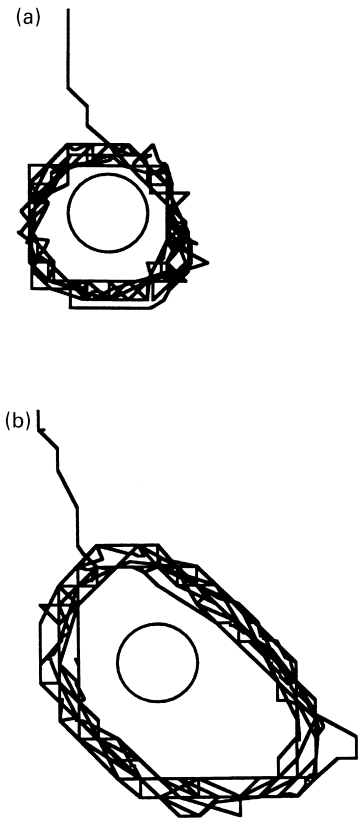


Fig. 16. Evolved circling: (a) circling that evolved in agents with feedforward networks; (b) circling that evolved in agents with recurrent networks.

behavior is to induce invariants, i.e., constraints, in the input space. Thus, through sensory–motor coordination the objects become discriminable because (a) data points belonging to one object are close together in data space (correlations), and (b) data belonging to different objects are far apart (large *GSI* values). These constraints in the input space are due to agent–environment interaction rather than neuronal processing only.

5. Discussion

The main goal of this paper was to investigate the implications of embedding neural networks in a physical structure, the body of a robot. Embedding a network in a body makes environmental, morphological and interactive constraints available for network design. In two case studies (Sections 3 and 4) we have shown how morphological constraints and constraints resulting from sensory–motor coordination can be used by embedded networks to solve the tasks of obstacle avoidance and category learning. The main reason to consider constraints is that real-world learning problems are often of type-2 and are thus intractable for traditional learning algorithms. One approach to deal with this problem is to increase the power of the learning algorithms. The alternative suggested in this paper is to focus on the nature of data themselves. In the case of embedded networks, the data are not just given by the designer, but are generated by the moving agent—and thus by the network—itself. By carefully studying how the data are generated, type-2 problems can be transformed into type-1 problems. This is the most fundamental point to be made in this paper. It fits well with the theory of neural networks: by incorporating constraints in the input data, the VC dimension of the network can be reduced which in turn leads to better generalization on small data sets (Abu-Mustafa, 1992; Meier, 1997). This is, of course, precisely what is needed in the real world where large training sets usually do not exist and learning has to occur after a few encounters with an object.

The work presented here is only a first step towards a better understanding of the relationship between different kinds of constraints and their implications for learning. What is needed in the future is a more formal approach that allows a more systematic description of constraints. For example, we argued that one important source of constraints is the ecological niche of an agent. If we want to exploit the constraints of an ecological niche systematically, we also need a systematic characterization of niches, a kind of taxonomy. This has turned out to be highly non-trivial. The problem is that such characterizations have to be done with respect to a particular agent, to its sensors and its motor system. The only properties of environments that matter are those that are behaviorally relevant. Let us take an example. To an ant small pebbles, twigs, and puddles are behaviorally relevant: it can sense them and avoid them, whereas to an elephant they are not: its sensory–motor system is not sufficiently fine-grained. Although a taxonomy would clearly be important, it has so far resisted clarification. Very few papers have ventured into this topic area. One approach is to define environments by the constraints they satisfy (Horswill, 1992, 1993). Horswill identified a number of so-called habitat constraints. An example is the ‘ground plane constraint’. An environment satisfies this constraint if all objects, including the agent, rest upon a planar surface. Another approach is to define environments by the predictability of the results of actions. Certain environments are more predictable than others. The less predictable an environment, the harder it is to design an agent for it. Thus, it would be desirable to have a characterization of environments in terms of their predictability from the agent’s point of view (see Wilson, 1991, for such an attempt). While these first attempts are promising, more research is needed to derive a taxonomy of niches that can be used for network design.

We showed that through mechanisms of sensory–motor coordination mobile agents can manipulate the world so as to impose constraints on their sensory data which simplify learning. The kind of sensory–motor coordination an agent can engage in depends on the complexity of its sensory–motor set-up (sensor and motor degrees of freedom). The

agents used in the experiments presented in this paper have a relatively limited sensory–motor system. It will thus be important to experiment with more sophisticated robots in order to study more complex kinds of agent–object interactions. The *GSI* provides a useful tool in this endeavor as it can be used to compare different architectures with respect to their success in transforming hard learning problems into benign ones. It will be of significant interest to compare, for example, different mechanisms of sensory–motor coordination with respect to their success in maximizing the *GSI* of a given learning problem. It will also be of interest to investigate how the morphology of a robot itself can be exploited. For example, different positions of the sensors with respect to each other, and with respect to the targets, might dramatically change the resulting learning performance. Finally, to come back to our initial claims, the exploitation of sensory-motor coordination, the coupling to the environment, furnishes an illustration of a mechanism for the focus-of-attention and the object constancy problems by providing the appropriate correlations and data transformations.

Acknowledgements

This research has been supported in part by grant number 11-50955.97 of the Swiss National Science Foundation.

References

Abu-Mustafa Y.S. (1992). Hints and the VC dimension. *Neural Computation, 5*, 278–288.

Almassy, N., Edelman, G.M., & Sporns, O. (1998). Behavioral constraints in the development of neuronal properties: a cortical model embedded in a real world device. *Cerebral Cortex, 8*, 346–361.

Arkin, R. (1998). *Behavior-based robotics*. Cambridge, MA: MIT Press.

Ashby, W.R. (1956). *An introduction to cybernetics*. London: Chapman and Hall.

Ballard D., Hayhoe M., Pook K., & Rao R. (1997). Deictic codes for the embodiment of cognition. *Behavioral and Brain Sciences, 23*, 233–265.

Bensinger D.G., Hayhoe M., & Ballard D. (1995). Visual memory in a natural task. *Investigative Ophthalamology and Visual Science, 36*, 14.

Brooks R.A. (1991). Intelligence without representation. *Artificial Intelligence, 47*, 139–160.

Chappell G.J., & Taylor J.G. (1993). The temporal Kohonen map. *Neural Networks, 6*, 441–445.

Chiel H.J., & Beer R.D. (1997). The brain has a body: adaptive behavior emerges from interactions of nervous system, body and environment. *Trends in Neuroscience, 20*, 553–557.

Clark A., & Thornton C. (1997). Trading spaces: computation, representation and the limits of uninformed learning. *Behavioural and Brain Sciences, 20*, 57–90.

Garcia, M., Chatterjee, A., Ruina, A., & Coleman, M. (in press). The simplest walking model: stability, complexity, and scaling. *ASME Journal of Biomechanical Engineering*.

Glenberg A.M. (1996). What memory is for?. *Behavioral and Brain Sciences, 20*, 1–55.

Hertz, J., Krogh, A., & Palmer, R.G. (1991). *Introduction to the theory of neural computation*. Redwood City, CA: Addison-Wesley.

Horswill, I. (1992). Characterizing adaptation by constraint. In F.J. Varela & P. Bourgine (Eds.), Toward a Practice of Autonomous Systems. *Proc First European Conference Artificial Life* (pp. 58–64). Cambridge, MA: MIT-Press.

Horswill, I. (1993). A simple, cheap, and robust visual navigation system. In J.-A. Meyer, H. L. Roitblat, & S. W. Wilson (Eds.), From Animals to Animats. *Proc. 2nd Int. Conf. Simulation of Adaptive Behavior* (pp. 129–136). Cambridge, MA: MIT Press (A Bradford Book).

Maris, M., & te Boekhorst, R. (1996). Exploiting physical constraints: heap formation through behavioral error in a group of robots. In *Proc. IROS'96, IEEE/RSJ International Conference on Intelligent Robots and Systems*.

Meier, D. (1997). Generalization and constraints in learning machines. Unpublished PhD thesis, Department of Computer Science, University of Zurich.

Millar, S. (1994). *Understanding and representing space*. Oxford: Oxford Science Publications.

Montello D.R., & Presson C.C. (1993). Movement and orientation in surrounding and imaginal spaces. (Manuscript cited in Glenberg, A. M. (1997)). What memory is for?. *Behavioral and Brain Sciences, 20*, 1–55.

Nolfi, S. (1996). Adaptation as a more powerful tool than decomposition and integration. Technical Report 96-03, Department of Neural Systems and Artificial Life, Institute of Psychology, C.N.R., Rome, Italy.

Pfeifer, R., & Scheier, C. (1997). Sensory–motor coordination: the metaphor and beyond. In R. Pfeifer & R. Brooks (Eds.), *Robotics and Autonomous Systems* (Special Issue on 'Practice and Future of Autonomous Agents'), 20 (pp. 157-178).

Pfeifer, R., & Scheier, C. (in press-a). *Understanding intelligence*. Cambridge, MA: MIT Press.

Pfeifer, R., & Scheier, C. (in press-b). Representation in natural and artificial agents: an embodied cognitive science perspective. *Zeitschrift fuer Naturforschung* [*Section C, A Journal of Biosciences*].

Pfeifer, R., & Verschure, P.F.M.J. (1992). Distributed adaptive control: a paradigm for designing autonomous agents. In F.J. Varela & P. Bourgine (Eds.), Toward a practice of autonomous systems. *Proc. First European Conference on Artificial Life* (pp. 21–30). Cambridge, MA: MIT Press.

Ruff H.A. (1984). Infants' manipulative exploration of objects: effects of age and object characteristics. *Development Psychology, 20*, 9–20.

Salinas E., & Romo R. (1998). Conversion of sensory signals into motor commands in primary cortex. *J. Neuroscience, 18 (10)*, 499–511.

Salomon, R. (1996). Neural networks in the context of autonomous agents: important concepts revisited. In C.H. Dagli, M. Akay, C.L.P. Chen, B.R. Fernández, & J. Ghosh (Eds.), *Proceedings of the Artificial Neural Networks in Engineering (ANNIE'96)* (pp. 109–116). New York: ASME Press.

Scheier, C., & Pfeifer, R. (1995). Classification as sensory-motor coordination. In *Proc. European Conference on Artificial Life, ECAL-95* (pp. 656-667).

Simons D.J., & Levin D.T. (1997). Change blindness. *Trends in Cognitive Sciences, 1*, 261–267.

Thelen, E., & Smith, L. (1994). *A dynamic systems approach to the development of cognition and action*. Cambridge, MA: MIT Press (Bradford Books).

Thornton, C. (1997). Separability is a learner's best friend. In J.A. Bullinaria, D.W. Glasspool, & G. Houghton (Eds.), *Proc. Fourth Neural Computation and Psychology Workshop: Connectionist representations* (pp. 40–47). London: Springer.

Thorpe, S.J., & Imbert, M. (1989). Biological constraints on connectionist models. In R. Pfeifer, Z. Schreter, & F. Fogelman-Soulie (Eds.), *Connectionism in perspective* (pp. 109–116). London: Wiley.

Vapnik V.N., & Chervonenkis A. (1989). On the uniform convergence of relative frequencies of events to their probabilities. *Theory Prob. Appl., 16,* 246–280.

Wilson, S.W. (1991). The animat path to AI. In J.-A. Meyer & S. W. Wilson (Eds.), From animals to animats. *Proc. of the 1st Int. Conf. on Simulation of Adaptive Behavior* (pp. 15–21). Cambridge, MA: MIT Press (A Bradford Book).